T07 CLAUDE.md : Bonnes Pratiques

Technique Intermediate

Écrire des instructions efficaces que Claude lit à chaque session

Ce qu'il faut inclure

Le CLAUDE.md efficace contient trois blocs essentiels : la stack technique, les commandes de build/test, et les conventions critiques. Tout ce que Claude peut déduire du code lui-même (structure de dossiers, framework détecté, package manager) n'a pas besoin d'être documenté.

```
## Stack
Next.js 14, TypeScript, PostgreSQL/Prisma, pnpm

## Commandes
- `pnpm dev`  — démarrer le serveur
- `pnpm test`  — lancer les tests
- `pnpm build` — build production

## Conventions critiques
- Toujours pnpm, jamais npm ni yarn
- Composants fonctionnels uniquement
- Fichiers kebab-case (mon-composant.tsx)
```

Ce qu'il ne faut PAS inclure

A éviter	Raison
Documentation narrative longue	Dilue les règles critiques
Contexte historique (« En 2023 on a décidé… »)	N'aide pas Claude à agir
Références à des fichiers entiers	Utiliser `@chemin/fichier` à la place
Règles que Claude respecte déjà	Bruit inutile, coût en tokens

Taille recommandée

Garder des instructions concises et vérifier leur effet sur des tâches réelles. Environ 200 lignes est un point de départ pratique, pas un seuil démontré de cohérence du modèle.

Règle pratique : si Claude fait la même erreur deux fois, ajouter la règle. Ne pas tout documenter préventivement.

Structure hiérarchique

```
~/.claude/CLAUDE.md          (global, tous projets)
/projet/CLAUDE.md            (projet, commité)
/projet/CLAUDE.local.md      (personnel, ajouter à .gitignore)
```

Dans un monorepo, Claude charge le CLAUDE.md racine puis fusionne le CLAUDE.md du sous-dossier actif. Les règles enfants complètent les règles parentes sans les effacer.

Import explicite avec @

```
# Dans CLAUDE.md
@docs/conventions/coding-standards.md
@docs/conventions/architecture.md
```

Les imports dans CLAUDE.md se chargent avec ce fichier d'instructions. Les règles générales se chargent largement ; les règles avec frontmatter `paths` s'appliquent lorsque les fichiers correspondants sont pertinents. Utiliser un skill si le workflow complet ne doit se charger qu'à l'invocation.

Architecture modulaire recommandée

```
.claude/
├── CLAUDE.md          (index, < 100 lignes)
└── rules/
    ├── testing.md     (seuils de couverture)
    ├── security.md    (invariants de sécurité)
    └── api.md         (conventions REST)
```

CLAUDE.md reste lisible d'un coup d'œil. Les règles détaillées résident dans `.claude/rules/` ; limiter les instructions propres aux fichiers avec `paths`.